Modernización de Infraestructura: De IIS On-Premises a OpenShift con GitOps

Nicolás Miretti

2026

Índice

1 Introduccion **3**
1.1 Contexto . . . 3
1.2 Problema . . . 4
1.3 Objetivos . . . 5
1.3.1 Objetivo general . . . 5
1.3.2 Objetivos especificos . . . 5
1.4 Alcance . . . 6
1.5 Metodologia . . . 6
1.6 Estructura del documento . . . 7

2 Marco Teorico **8**
2.1 Infraestructura tradicional . . . 8
2.1.1 Servidores fisicos . . . 8
2.1.2 Maquinas virtuales . . . 9
2.1.3 Servidores web e IIS . . . 10
2.1.3.1 IIS en detalle . . . 10
2.1.3.2 El modelo de despliegue tipico con IIS . . . 11
2.1.3.3 Limitaciones de IIS como plataforma . . . 12
2.2 Contenedores . . . 13
2.2.1 De servidores fisicos a maquinas virtuales a contenedores . . . 13
2.2.2 Que es un contenedor . . . 14
2.2.3 Docker y el estandar OCI . . . 14
2.2.3.1 Ejemplo: de IIS a contenedor . . . 15
2.2.3.2 El estandar OCI . . . 16
2.2.4 Contenedores vs IIS: comparacion directa . . . 17
2.3 Orquestacion de contenedores . . . 18
2.3.1 Por que no alcanza con contenedores solos . . . 18
2.3.2 Kubernetes . . . 19
2.3.2.1 Que problemas resuelve . . . 19
2.3.2.2 Conceptos fundamentales de Kubernetes . . . 20
2.3.2.3 El modelo declarativo . . . 22
2.3.3 Red Hat OpenShift . . . 22
2.3.3.1 OpenShift vs Kubernetes: cuando y por que . . . 24
2.4 Modelos de despliegue de infraestructura . . . 25
2.4.1 On-premises . . . 25
2.4.2 Cloud publica . . . 26
2.4.3 Modelo hibrido . . . 26
2.4.3.1 ARO - Azure Red Hat OpenShift . . . 27
2.5 Conceptos fundamentales de desarrollo y operaciones . . . 28
2.5.1 Control de versiones y Git . . . 28

2.5.2 YAML . . . 29
2.5.3 Entornos de despliegue . . . 30
2.5.4 DevOps . . . 31
2.5.5 Monolitos y microservicios . . . 32
2.6 Practicas de entrega de software . . . 34
2.6.1 Integracion Continua y Entrega Continua (CI/CD) . . . 34
2.6.1.1 Pipelines . . . 35
2.6.2 GitOps . . . 35
2.6.2.1 Principios fundamentales . . . 36
2.6.2.2 Por que Git como fuente de verdad . . . 36
2.6.2.3 Push vs Pull model . . . 37
2.6.2.4 GitOps vs despliegue manual: un ejemplo . . . 37
2.6.3 Infraestructura como Codigo (IaC) . . . 38
2.7 Herramientas clave . . . 39
2.7.1 Argo CD . . . 39
2.7.1.1 Como funciona . . . 39
2.7.1.2 Conceptos principales . . . 40
2.7.1.3 Interfaz y usabilidad . . . 40
2.7.2 Helm . . . 41
2.7.2.1 El problema que resuelve . . . 41
2.7.2.2 Estructura de un Helm chart . . . 42
2.7.2.3 Ejemplo conceptual . . . 42
2.7.3 Kustomize . . . 43
2.7.3.1 Enfoque: bases y overlays . . . 43
2.7.3.2 Helm vs Kustomize . . . 44
2.7.4 Red Hat Developer Hub . . . 45
2.7.4.1 El problema que resuelve . . . 45
2.7.4.2 Capacidades principales . . . 46
2.7.4.3 Rol en la modernizacion . . . 47
2.7.5 Ansible . . . 47
2.7.5.1 Caracteristicas principales . . . 47
2.7.5.2 Ejemplo de playbook . . . 48
2.7.5.3 Ansible en el contexto de OpenShift . . . 49
2.8 Patrones de arquitectura . . . 49
2.8.1 Patron App of Apps . . . 49
2.8.1.1 El problema . . . 50
2.8.1.2 La solucion . . . 50
2.8.1.3 Beneficios . . . 51
2.8.1.4 Variantes . . . 51
2.8.2 Operadores de Kubernetes . . . 52
2.8.2.1 El problema que resuelven . . . 52
2.8.2.2 Como funcionan . . . 52
2.8.2.3 Ejemplo . . . 53
2.9 Resumen del capitulo . . . 53

Capítulo 1

Introduccion

1.1 Contexto

En los ultimos años, las organizaciones de tecnologia han enfrentado una presion creciente por acelerar la entrega de software, mejorar la confiabilidad de sus sistemas y reducir los costos operativos asociados a infraestructuras tradicionales. Lo que antes era aceptable, como servidores configurados manualmente, despliegues coordinados por correo electronico y procesos que dependian del conocimiento de unas pocas personas, hoy representa un riesgo operativo y una barrera para la competitividad.

La empresa objeto de este trabajo no es ajena a esta realidad. Durante años, su infraestructura de aplicaciones se sustento sobre **Internet Information Services (IIS)** corriendo en servidores **Windows Server on-premises**, con procesos de despliegue en su mayoria manuales y herramientas de CI/CD que fueron quedando obsoletas. Este modelo, si bien funcional en su momento, presentaba limitaciones cada vez mas evidentes: tiempos de despliegue prolongados, falta de estandarizacion entre equipos, dificultad para escalar, y una fuerte dependencia de conocimiento no documentado.

Frente a este escenario, la organizacion inicio un proceso de modernizacion que busca transformar la forma en que se construye, despliega y opera el software. No se trato de un cambio instantaneo ni de una migracion completa a la nube publica, sino de una **transicion progresiva**

hacia un modelo hibrido basado en **Red Hat OpenShift** (tanto on-premises como en Azure a traves de **ARO - Azure Red Hat OpenShift**), incorporando practicas de **GitOps**, **infraestructura como codigo** y herramientas modernas de automatizacion.

1.2 Problema

El modelo de infraestructura tradicional de la empresa presentaba una serie de problemas que impactaban tanto en la operacion diaria como en la capacidad de evolucion tecnologica:

- **Deploys manuales y propensos a error**: los despliegues en IIS requerian intervenciones manuales, lo que incrementaba el riesgo de errores humanos y hacia que cada puesta en produccion fuera un evento de alto estres.
- **Falta de estandarizacion**: cada equipo o aplicacion podia tener un proceso de despliegue diferente, dificultando la operacion, el soporte y la incorporacion de nuevos miembros.
- **Herramientas de CI/CD obsoletas**: las herramientas existentes no se adaptaban a las necesidades actuales ni permitian implementar practicas modernas de entrega continua.
- **Conocimiento concentrado**: los procedimientos dependian del conocimiento de pocas personas, generando cuellos de botella y riesgo ante rotacion de personal.
- **Escalabilidad limitada**: escalar aplicaciones o incorporar nuevos equipos a la plataforma era un proceso lento y costoso.
- **Sin versionado de la infraestructura**: no existia un registro claro de que estaba desplegado, en que version, ni como reproducir un entorno.

Estos problemas no eran exclusivos del area de aplicaciones web. La empresa tambien operaba herramientas como **Control-M** para scheduling de procesos batch, **aplicaciones Java**

legacy y **procesos de RPA (Robotic Process Automation)**, todos en proceso de evaluacion o modernizacion. Sin embargo, el alcance de esta tesis se centra en el eje principal de la transformacion: **la plataforma de contenedores y el modelo de entrega de software**.

1.3 Objetivos

1.3.1 Objetivo general

Documentar y analizar el proceso de modernizacion de la infraestructura de entrega de software de una empresa, desde un modelo basado en IIS on-premises con despliegues manuales, hacia una plataforma basada en Red Hat OpenShift con practicas de GitOps, infraestructura como codigo y automatizacion.

1.3.2 Objetivos especificos

1. Describir la situacion inicial de la infraestructura y los procesos de la empresa, identificando sus limitaciones.
2. Presentar los conceptos teoricos necesarios para comprender las tecnologias y practicas adoptadas.
3. Documentar la evolucion del proceso de modernizacion a traves de sus distintas etapas, desde los primeros comandos manuales hasta la implementacion de GitOps con Argo CD.
4. Analizar las decisiones tecnicas tomadas en cada etapa y sus justificaciones.
5. Evaluar los resultados obtenidos, comparando el estado anterior con el actual en terminos de eficiencia, estandarizacion y autonomia de los equipos.

1.4 Alcance

Esta tesis cubre los siguientes aspectos:

Dentro del alcance: - El proceso de adopcion de Red Hat OpenShift como plataforma de contenedores (on-premises y ARO). - La evolucion de los procesos de despliegue: desde comandos manuales, pasando por scripts, pipelines, hasta GitOps con Argo CD. - La implementacion de infraestructura como codigo con Helm y Kustomize. - La adopcion del patron App of Apps para la gestion declarativa de aplicaciones. - La incorporacion de herramientas complementarias: Red Hat Developer Hub y Ansible. - El impacto organizacional del cambio en los equipos de desarrollo y operaciones.

Fuera del alcance: - La modernizacion de herramientas de scheduling (Control-M). - La migracion de aplicaciones Java legacy. - Los procesos de RPA. - Detalles de la arquitectura interna de las aplicaciones (microservicios, codigo fuente, etc.). - Aspectos comerciales o financieros de la transformacion.

1.5 Metodologia

El enfoque de esta tesis es **descriptivo y basado en la experiencia directa** del autor como parte del equipo que llevo adelante la modernizacion. Se documenta el proceso tal como ocurrio, con sus aciertos, errores y decisiones iterativas.

La metodologia incluye:

- **Observacion participante**: el autor fue parte activa del proceso de transformacion.
- **Documentacion retrospectiva**: se reconstruyen las etapas del proceso a partir de documenta-

cion interna, registros de configuracion y memoria del equipo.

- **Comparacion antes/despues**: cada etapa se contrasta con el estado previo para evidenciar las mejoras o los desafios introducidos.
- **Ejemplos tecnicos reales**: se incluyen configuraciones, manifiestos y capturas (sanitizados) para ilustrar los conceptos con casos concretos.

1.6 Estructura del documento

El presente documento se organiza de la siguiente manera:

- **Capitulo 1 - Introduccion** (este capitulo): presenta el contexto, el problema, los objetivos y el alcance.
- **Capitulo 2 - Marco Teorico**: introduce los conceptos fundamentales necesarios para comprender las tecnologias y practicas adoptadas.
- **Capitulo 3 - Situacion Inicial**: describe en detalle la arquitectura y los procesos de la empresa antes de la modernizacion.
- **Capitulo 4 - Proceso de Modernizacion**: documenta cada etapa de la transicion con detalle tecnico y decisiones tomadas.
- **Capitulo 5 - Arquitectura Actual**: presenta la plataforma resultante y su funcionamiento.
- **Capitulo 6 - Resultados y Analisis**: compara el antes y el despues, evaluando las mejoras logradas.
- **Capitulo 7 - Conclusiones y Trabajo Futuro**: reflexiones finales y lineas de trabajo pendientes.

Capítulo 2

Marco Teorico

Este capitulo presenta los conceptos fundamentales necesarios para comprender las tecnologias, practicas y decisiones que se describen en el resto de la tesis. El objetivo no es ser una referencia exhaustiva de cada herramienta, sino brindar al lector una base solida para entender el proceso de modernizacion y las razones detras de cada eleccion tecnologica.

El capitulo comienza describiendo la infraestructura tradicional (el punto de partida de esta tesis), para luego ir introduciendo progresivamente los conceptos y tecnologias que conforman la plataforma moderna.

2.1 Infraestructura tradicional

Antes de hablar de contenedores, Kubernetes o GitOps, es necesario entender como funciona la infraestructura que se busca modernizar. Esta seccion describe los componentes y conceptos del modelo tradicional de TI que la empresa utilizo durante años.

2.1.1 Servidores fisicos

Un **servidor** es, en esencia, una computadora diseñada para ejecutar aplicaciones y ofrecer servicios a otros sistemas o usuarios. A diferencia de una PC de escritorio, un servidor esta

optimizado para funcionar las 24 horas, los 7 dias de la semana, con hardware redundante (fuentes de alimentacion, discos, ventilacion) y mayor capacidad de procesamiento y memoria.

En el modelo tradicional, cada aplicacion se instala directamente sobre el sistema operativo del servidor. Si una empresa tiene 10 aplicaciones, podria necesitar 10 servidores (o al menos varios, agrupando aplicaciones compatibles). Este modelo tiene problemas evidentes:

- **Desperdicio de recursos**: un servidor que usa solo el 10% de su capacidad sigue consumiendo energia, espacio y mantenimiento como si usara el 100%.
- **Rigidez**: agregar un nuevo servidor requiere comprarlo, instalarlo fisicamente en el data center, configurar red y sistema operativo. Un proceso que puede llevar semanas o meses.
- **Acoplamiento**: si el servidor falla, todas las aplicaciones que corren en el caen.

2.1.2 Maquinas virtuales

Las **maquinas virtuales (VMs)** surgieron como solucion al problema del desperdicio de recursos. Una VM es una emulacion por software de una computadora completa, con su propio sistema operativo, memoria y almacenamiento virtual, que corre dentro de un servidor fisico.

Un **hipervisor** (como VMware vSphere, Hyper-V o KVM) es el software que permite ejecutar multiples VMs sobre un mismo servidor fisico, repartiendo los recursos de hardware entre ellas.

Beneficios sobre los servidores fisicos: - **Consolidacion**: un unico servidor fisico puede alojar 5, 10 o mas VMs, aprovechando mejor los recursos. - **Aislamiento**: si una VM falla, las demas no se ven afectadas. - **Flexibilidad**: crear una nueva VM toma minutos u horas, no semanas. - **Snapshots**: se puede capturar el estado de una VM en un punto en el tiempo y restaurarlo si algo

sale mal.

Limitaciones: - **Peso**: cada VM incluye un sistema operativo completo. Si se necesitan 10 VMs con Windows Server, se estan ejecutando 10 copias del sistema operativo, cada una consumiendo memoria y disco. - **Tiempo de arranque**: iniciar una VM lleva minutos, ya que debe arrancar el sistema operativo completo. - **Licenciamiento**: cada VM con Windows requiere su propia licencia del sistema operativo. - **Portabilidad limitada**: mover una VM de un hipervisor a otro (por ejemplo, de VMware a Hyper-V) no es trivial.

2.1.3 Servidores web e IIS

Un **servidor web** es un software que recibe solicitudes de los usuarios (tipicamente a traves del protocolo HTTP/HTTPS desde un navegador) y responde con el contenido solicitado: paginas HTML, datos de una API, archivos, etc. Es el intermediario entre el usuario final y la aplicacion.

Los servidores web mas utilizados en la industria son:

- **Apache HTTP Server**: de codigo abierto, dominante durante los años 2000.
- **Nginx**: de codigo abierto, popular por su rendimiento y eficiencia con alto trafico.
- **IIS (Internet Information Services)**: el servidor web de Microsoft, integrado en Windows Server.

2.1.3.1 IIS en detalle

IIS es el servidor web que Microsoft incluye como componente de **Windows Server**. Es la plataforma sobre la cual se despliegan aplicaciones desarrolladas con tecnologias Microsoft, como **ASP.NET**, **ASP.NET Core**, y servicios **WCF (Windows Communication Foundation)**.

Caracteristicas principales de IIS:

- **Integrado con el ecosistema Microsoft**: se administra desde el sistema operativo, se integra con Active Directory para autenticacion, y soporta nativamente aplicaciones .NET.
- **Application Pools**: mecanismo de aislamiento que permite ejecutar multiples aplicaciones en un mismo servidor IIS de forma independiente. Si una aplicacion falla, las demas siguen funcionando.
- **Interfaz grafica (IIS Manager)**: herramienta visual para configurar sitios, bindings (que puertos y dominios escucha), certificados SSL, etc.
- **Configuracion basada en archivos**: la configuracion de IIS se almacena en archivos XML (`web.config`, `applicationHost.config`), lo que permite versionarla.

2.1.3.2 El modelo de despliegue tipico con IIS

En una empresa que usa IIS, el proceso de despliegue de una aplicacion tipicamente luce asi:

1. Un desarrollador compila la aplicacion y genera un paquete de despliegue (archivos DLL, configuraciones, assets).
2. El paquete se copia manualmente (o con un script basico) al servidor donde corre IIS.
3. Se configura un **sitio** en IIS Manager: se indica la carpeta donde estan los archivos, el puerto, el dominio.
4. Se configura el **Application Pool**: version de .NET, identidad bajo la que corre, limites de memoria.
5. Se prueban los cambios accediendo a la aplicacion desde el navegador.

Este proceso, repetido para cada aplicacion y cada entorno (desarrollo, testing, produccion), genera una carga operativa significativa y es propenso a inconsistencias: “en el servidor de desarrollo funciona, pero en produccion no, porque la configuracion es diferente”.

2.1.3.3 Limitaciones de IIS como plataforma

Si bien IIS es una tecnologia madura y confiable, presenta limitaciones cuando se la evalua como plataforma para operar decenas o cientos de aplicaciones:

- **Acoplado a Windows**: IIS solo corre sobre Windows Server, lo que implica licencias de sistema operativo por cada servidor.
- **Escalado manual**: escalar una aplicacion requiere configurar manualmente mas servidores con IIS y un balanceador de carga externo.
- **Sin auto-recuperacion**: si una aplicacion se cae, IIS puede reiniciar el Application Pool, pero no puede mover la aplicacion a otro servidor automaticamente.
- **Configuracion no estandarizada**: cada servidor puede tener configuraciones ligeramente diferentes, acumuladas a lo largo del tiempo (configuration drift).
- **Despliegues acoplados al servidor**: la aplicacion esta atada al servidor donde fue instalada. Moverla a otro servidor requiere repetir la configuracion.

Estas limitaciones son manejables con pocas aplicaciones, pero se amplifican a medida que la organizacion crece. Es en este punto donde tecnologias como los contenedores empiezan a ser relevantes.

2.2 Contenedores

2.2.1 De servidores fisicos a maquinas virtuales a contenedores

La historia de la infraestructura de TI puede verse como una busqueda progresiva de **mayor eficiencia y aislamiento**:

- **Servidores fisicos**: una aplicacion por servidor. Maximo aislamiento, minima eficiencia.
- **Maquinas virtuales**: multiples sistemas operativos virtuales en un servidor fisico. Buen aislamiento, mejor eficiencia. Pero cada VM arrastra un sistema operativo completo.
- **Contenedores**: multiples aplicaciones aisladas que **comparten el kernel del sistema operativo** del host. Excelente eficiencia, aislamiento suficiente para la mayoria de los casos.

La diferencia fundamental es lo que se empaqueta:

Caracteristica	VM	Contenedor
Incluye sistema operativo	Si (completo)	No (comparte kernel del host)
Tamaño tipico	Gigabytes	Megabytes
Tiempo de arranque	Minutos	Segundos
Aislamiento	Hardware virtualizado	Procesos aislados (namespaces, cgroups)
Portabilidad	Limitada al hipervisor	Alta (cualquier runtime compatible)
Densidad	Decenas por servidor	Cientos por servidor

2.2.2 Que es un contenedor

Un **contenedor** es una unidad de software que empaqueta el codigo de una aplicacion junto con todas sus dependencias (librerias, runtime, configuracion, archivos necesarios) de manera que pueda ejecutarse de forma **consistente en cualquier entorno**.

Para entenderlo con una analogia concreta: en el modelo IIS, desplegar una aplicacion es como mudarse a una casa nueva llevando solo los muebles, esperando que la casa tenga la electricidad, el agua y el gas que uno necesita. Si la instalacion electrica es diferente, los muebles no "funcionan". Un contenedor es como mudarse con toda la casa incluida: muebles, electricidad, agua, gas. Funciona igual en cualquier terreno.

Tecnicamente, un contenedor logra este aislamiento usando funcionalidades del kernel de Linux:

- **Namespaces**: proveen aislamiento de procesos, red, sistema de archivos y usuarios. Cada contenedor "ve" su propio entorno, como si fuera la unica aplicacion corriendo.
- **Cgroups (Control Groups)**: limitan los recursos (CPU, memoria, disco) que un contenedor puede usar, evitando que una aplicacion consuma todos los recursos del host.
- **Union File Systems**: permiten construir imagenes en capas reutilizables, reduciendo el espacio en disco y el tiempo de descarga.

2.2.3 Docker y el estandar OCI

Docker, lanzado en 2013, fue la herramienta que popularizo el concepto de contenedores. Antes de Docker, tecnologias como LXC (Linux Containers) ya existian, pero Docker simplifico drasticamente la experiencia del desarrollador al introducir:

- **Dockerfile**: un archivo de texto que describe paso a paso como construir una imagen de contenedor. Es una receta reproducible.
- **Imagenes**: paquetes inmutables (que no cambian una vez creados) que contienen todo lo necesario para ejecutar una aplicacion. Se construyen en capas, lo que permite reutilizar componentes comunes entre aplicaciones.
- **Registros de imagenes**: repositorios centralizados (como Docker Hub, Quay.io o registros privados) donde se almacenan y distribuyen las imagenes. Funcionan de manera similar a como un repositorio de codigo almacena codigo fuente.
- **Docker Engine**: el motor que ejecuta los contenedores en un host.

2.2.3.1 Ejemplo: de IIS a contenedor

Para ilustrar la diferencia, consideremos el despliegue de una aplicacion web .NET:

Con IIS (modelo tradicional): 1. Provisionar un servidor Windows con IIS instalado. 2. Instalar la version correcta de .NET Framework o .NET Runtime. 3. Copiar los archivos de la aplicacion al servidor. 4. Configurar el sitio en IIS (puerto, dominio, Application Pool). 5. Verificar que todo funcione. 6. Repetir para cada entorno (desarrollo, staging, produccion).

Con contenedores: 1. Escribir un Dockerfile que define: imagen base con .NET, copiar archivos de la aplicacion, exponer el puerto. 2. Construir la imagen (un solo comando). 3. Ejecutar la imagen en cualquier entorno (un solo comando). 4. La aplicacion funciona exactamente igual en todos los entornos.

El Dockerfile podria verse asi:

```
# Imagen base con .NET Runtime
```

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0


# Copiar la aplicacion compilada
COPY ./publish /app


# Definir el directorio de trabajo
WORKDIR /app


# Exponer el puerto
EXPOSE 8080


# Comando para iniciar la aplicacion
ENTRYPOINT ["dotnet", "MiAplicacion.dll"]
```

Este archivo de 6 lineas reemplaza un proceso manual de multiples pasos y garantiza que la aplicacion se ejecute de la misma forma en cualquier lugar donde se ejecute el contenedor.

2.2.3.2 El estandar OCI

Con el crecimiento del ecosistema, surgio la necesidad de estandarizar el formato de contenedores para no depender exclusivamente de Docker. La **Open Container Initiative (OCI)**, fundada en 2015 bajo la Linux Foundation, definio dos especificaciones clave:

- **Image Spec**: como se construye y estructura una imagen de contenedor.
- **Runtime Spec**: como se ejecuta un contenedor.

Esto garantiza que las imagenes construidas con cualquier herramienta compatible con OCI (Docker, Podman, Buildah) puedan ejecutarse en cualquier runtime compatible. En la practica, esto significa que la organizacion no queda atada a Docker como unica herramienta.

2.2.4 Contenedores vs IIS: comparacion directa

Para sintetizar la diferencia entre el modelo basado en IIS y el modelo basado en contenedores:

Aspecto	IIS tradicional	Contenedores
Unidad de despliegue	Archivos copiados a un servidor con IIS	Imagen de contenedor autosuficiente
Dependencias	Instaladas en el servidor (pueden variar)	Incluidas en la imagen (siempre iguales)
Consistencia entre entornos	Baja (cada servidor puede diferir)	Alta (misma imagen en todos lados)
Sistema operativo	Windows Server obligatorio	Cualquier host con runtime de contenedores
Tiempo de despliegue	Minutos a horas (manual)	Segundos (automatizado)
Escalado	Manual (nuevo servidor + IIS + config)	Automatico (nueva instancia del contenedor)
Aislamiento	Application Pools (dentro del mismo OS)	Contenedores separados (kernel aislado)

Aspecto	IIS tradicional	Contenedores
Portabilidad	Baja (atado al servidor)	Alta (corre en cualquier host compatible)
Versionado	Archivos en el servidor (dificil de rastrear)	Imagenes versionadas en un registro
Rollback	Complejo (restaurar archivos anteriores)	Simple (ejecutar la imagen de la version anterior)

Esta tabla resume por que muchas organizaciones estan migrando de plataformas como IIS hacia contenedores: no porque IIS sea una mala tecnologia, sino porque el modelo de contenedores resuelve problemas estructurales de escalabilidad, consistencia y operacion que las plataformas tradicionales no fueron diseñadas para manejar.

2.3 Orquestacion de contenedores

2.3.1 Por que no alcanza con contenedores solos

Los contenedores resuelven el problema de empaquetar y ejecutar una aplicacion de forma consistente. Pero una organizacion no tiene una sola aplicacion: tiene decenas o cientos, cada una con multiples instancias, diferentes necesidades de red, almacenamiento y configuracion.

Gestionar todo esto manualmente se vuelve insostenible rapidamente:

- ¿En que servidor ejecuto cada contenedor?

- ¿Que pasa si un servidor se cae y tenia 20 contenedores?
- ¿Como hago para que los contenedores se comuniquen entre si?
- ¿Como expongo una aplicacion a los usuarios finales?
- ¿Como actualizo una aplicacion sin downtime?
- ¿Como escalo una aplicacion si recibe mucho trafico?

Responder a todas estas preguntas manualmente es exactamente el tipo de trabajo operativo que la modernizacion busca eliminar. Para eso existen los **orquestadores de contenedores**.

2.3.2 Kubernetes

Kubernetes (frecuentemente abreviado como K8s) es una plataforma de orquestacion de contenedores de codigo abierto, originalmente desarrollada por Google y donada a la **Cloud Native Computing Foundation (CNCF)** en 2015. Nacio de la experiencia interna de Google con sistemas como Borg y Omega, que gestionaban millones de contenedores en sus data centers.

Hoy, Kubernetes es el estandar de la industria para orquestacion de contenedores. La gran mayoria de las plataformas de contenedores empresariales (incluyendo OpenShift) estan construidas sobre Kubernetes.

2.3.2.1 Que problemas resuelve

- **Scheduling**: decide en que nodo (servidor) del cluster se ejecuta cada contenedor, en funcion de los recursos disponibles. El administrador no necesita elegir manualmente donde corre cada aplicacion.
- **Alta disponibilidad**: si un contenedor o un nodo falla, Kubernetes automaticamente reprograma las cargas de trabajo en nodos sanos. La aplicacion se recupera sin intervencion

humana.

- **Escalado**: permite escalar aplicaciones horizontalmente (mas instancias) o verticalmente (mas recursos), de forma manual o automatica en base a metricas como uso de CPU o memoria.
- **Networking**: proporciona un modelo de red donde cada pod tiene su propia direccion IP y los servicios se descubren automaticamente. Los desarrolladores no necesitan preocuparse por IPs o puertos especificos.
- **Gestion de configuracion y secretos**: permite inyectar configuraciones y credenciales en los contenedores de forma segura, sin incluirlas en la imagen.
- **Rolling updates y rollbacks**: permite actualizar aplicaciones de forma progresiva (sin downtime) y revertir cambios si algo falla.

2.3.2.2 Conceptos fundamentales de Kubernetes

Para comprender el resto de esta tesis, es necesario conocer los siguientes conceptos:

Cluster y nodos: - **Cluster**: conjunto de servidores (nodos) que trabajan juntos bajo la gestion de Kubernetes. - **Nodo master (plano de control)**: ejecuta los componentes que toman decisiones sobre el cluster (donde correr los contenedores, como responder a fallos, etc.). - **Nodo worker**: ejecuta las aplicaciones. Es donde realmente corren los contenedores.

Recursos basicos: - **Pod**: la unidad minima de despliegue en Kubernetes. Un pod puede contener uno o mas contenedores que comparten red y almacenamiento. En la practica, la mayoria de los pods contienen un solo contenedor. - **Deployment**: recurso que define el estado deseado de una aplicacion: que imagen usar, cuantas replicas ejecutar, como actualizarse. Kubernetes se encarga de mantener ese estado. - **Service**: abstraccion de red que expone un conjunto de pods como

un servicio estable. Aunque los pods se creen y destruyan constantemente, el Service provee una direccion fija para acceder a la aplicacion. - **Namespace**: mecanismo de aislamiento logico dentro de un cluster. Permite separar recursos por equipo, proyecto o entorno (por ejemplo, un namespace para “desarrollo” y otro para “produccion”).

Configuracion: - **ConfigMap**: recurso para almacenar configuracion no sensible (variables de entorno, archivos de configuracion) que se inyecta en los pods. - **Secret**: similar a ConfigMap, pero diseñado para datos sensibles (contraseñas, tokens, certificados). Se almacena codificado en base64 y puede integrarse con sistemas de gestion de secretos externos.

Acceso externo: - **Ingress**: recurso que define reglas para enrutar trafico HTTP/HTTPS externo hacia los servicios internos del cluster. Funciona como un punto de entrada que decide a que aplicacion enviar cada solicitud segun el dominio o la ruta.

Conceptos de red basicos:

Para entender como Kubernetes gestiona la comunicacion, es util repasar algunos conceptos basicos de redes:

- **Puerto**: un numero (0-65535) que identifica un servicio especifico en un servidor. Por ejemplo, el puerto 80 se usa para HTTP, el 443 para HTTPS.
- **DNS (Domain Name System)**: sistema que traduce nombres legibles (como `mi-app.empresa.com`) a direcciones IP. Kubernetes incluye un DNS interno para que los servicios se encuentren por nombre.
- **Balanceo de carga**: tecnica que distribuye el trafico entrante entre multiples instancias de una aplicacion para evitar sobrecargar una sola instancia.

- **TLS/SSL**: protocolos de cifrado que aseguran la comunicacion entre el usuario y la aplicacion (el “candadito” en el navegador).

2.3.2.3 El modelo declarativo

Un aspecto fundamental de Kubernetes es su **modelo declarativo**. En lugar de dar instrucciones paso a paso (“crea un contenedor, conectalo a esta red, exponelo en este puerto”), el usuario **declara el estado deseado** en un archivo, y Kubernetes se encarga de alcanzar y mantener ese estado.

Por ejemplo, un Deployment que declara “quiero 3 replicas de mi aplicacion” hara que Kubernetes: - Cree 3 pods si no existen. - Recree un pod si uno se cae. - Elimine pods sobrantes si por alguna razon hay mas de 3.

Este modelo es radicalmente diferente al modelo imperativo de IIS, donde el administrador debe ejecutar cada accion manualmente y verificar el resultado. En Kubernetes, el administrador describe **que quiere** y la plataforma se encarga del **como**.

Este modelo declarativo es la base sobre la que se construyen practicas como GitOps e infraestructura como codigo, que se describen mas adelante en este capitulo.

2.3.3 Red Hat OpenShift

Red Hat OpenShift es una plataforma de contenedores empresarial construida sobre Kubernetes. Si Kubernetes es el motor, OpenShift es el vehiculo completo: agrega todo lo necesario para que una organizacion pueda operar contenedores en produccion de forma segura y eficiente.

OpenShift toma Kubernetes como base y le añade:

- **Seguridad reforzada por defecto**: los contenedores corren con permisos restringidos mediante un mecanismo llamado **Security Context Constraints (SCC)**. Por defecto, los contenedores no pueden ejecutarse como usuario root (administrador), lo que reduce significativamente la superficie de ataque.
- **RBAC extendido (Role-Based Access Control)**: sistema de permisos granular que define **quien** puede hacer **que** dentro del cluster. Por ejemplo, un desarrollador puede ver los logs de su aplicacion pero no puede modificar la configuracion del cluster. Un administrador puede hacer ambas cosas. Esto es fundamental en organizaciones donde multiples equipos comparten la misma plataforma.
- **Consola web integrada**: una interfaz grafica completa que permite a desarrolladores y operadores gestionar recursos, ver logs, monitorear metricas y administrar el cluster sin necesidad de usar la linea de comandos para todo.
- **Routes**: extension de Kubernetes que simplifica la exposicion de aplicaciones al exterior con soporte nativo de TLS. Mientras que en Kubernetes vanilla se necesita configurar un Ingress Controller y crear recursos Ingress, en OpenShift las Routes son ciudadanos de primera clase.
- **Builds integrados (Source-to-Image)**: capacidad de construir imagenes de contenedores directamente desde el codigo fuente, sin necesidad de escribir un Dockerfile. Util para equipos que recien comienzan con contenedores.
- **OperatorHub**: catalogo integrado de **operadores** (componentes que se explican en la seccion 2.8.2) para instalar y gestionar servicios complejos como bases de datos, colas de mensajes o herramientas de monitoreo.
- **Soporte empresarial**: respaldo de Red Hat con soporte tecnico, actualizaciones de seguridad, ciclos de vida definidos y certificacion de compatibilidad.

2.3.3.1 OpenShift vs Kubernetes: cuando y por que

Es comun preguntarse por que usar OpenShift en lugar de Kubernetes directamente. La diferencia principal es el **nivel de abstraccion y soporte**:

Aspecto	Kubernetes vanilla	Red Hat OpenShift
Instalacion	Manual, multiples opciones	Instalador integrado
Seguridad	Configurable, responsabilidad del usuario	Hardened por defecto (SCC, RBAC)
Consola web	Dashboard basico (opcional)	Consola completa integrada
Networking	Requiere instalar un plugin de red	SDN integrado (OVN-Kubernetes)
Builds	No incluido	Source-to-Image, Buildah
Soporte	Comunidad	Red Hat (SLA empresarial)
Actualizaciones	Manuales	Over-the-air (OTA) gestionadas
Costo	Gratuito (solo infraestructura)	Licencia de suscripcion

Para una empresa que necesita operar en produccion con garantias de soporte, seguridad y estabilidad, OpenShift reduce significativamente la complejidad operativa comparado con administrar Kubernetes por cuenta propia. El costo de la suscripcion se compensa con la reduccion de esfuerzo operativo y el acceso a soporte empresarial.

2.4 Modelos de despliegue de infraestructura

La decision sobre **donde** ejecutar la infraestructura es tan importante como la decision de **que** tecnologias usar. Existen tres modelos principales, cada uno con sus ventajas, desventajas y casos de uso.

2.4.1 On-premises

El modelo **on-premises** (literalmente "en las instalaciones") significa que la infraestructura se ejecuta en servidores fisicos ubicados en un data center propio o contratado por la organizacion. La empresa es responsable de todo: hardware, red, almacenamiento, sistema operativo, actualizaciones, seguridad fisica y logica.

Es el modelo que la mayoria de las empresas utilizaron historicamente y el punto de partida de esta tesis.

Ventajas: - **Control total**: la organizacion tiene control completo sobre el hardware, la red y los datos. - **Cumplimiento normativo**: en industrias reguladas (banca, salud, gobierno), mantener los datos on-premises puede ser un requisito legal o regulatorio. - **Latencia predecible**: las aplicaciones que necesitan comunicarse con sistemas internos tienen latencia minima al estar en la misma red. - **Sin dependencia de terceros**: no hay riesgo de cambios de precios o condiciones por parte de un proveedor cloud.

Desventajas: - **Inversion de capital (CapEx)**: requiere comprar y mantener hardware, lo que implica una inversion inicial significativa. - **Escalabilidad limitada**: escalar implica comprar mas hardware, lo que lleva semanas o meses. - **Responsabilidad operativa**: el equipo de TI debe

gestionar todo el stack, desde el hardware hasta la plataforma. - **Riesgo de obsolescencia**: el hardware se deprecia y requiere renovacion periodica.

2.4.2 Cloud publica

En el modelo de **cloud publica**, la infraestructura se ejecuta en servidores de un proveedor externo (como Microsoft Azure, Amazon Web Services o Google Cloud Platform). El proveedor gestiona el hardware, la red y, en muchos casos, parte del software base.

Ventajas: - **Elasticidad**: se pueden aprovisionar o liberar recursos en minutos segun la demanda. - **Modelo de costos operativo (OpEx)**: se paga por lo que se usa, sin inversion inicial en hardware. - **Servicios gestionados**: el proveedor ofrece servicios como bases de datos, colas de mensajes, balanceadores, etc., reduciendo la carga operativa. - **Alcance global**: permite desplegar aplicaciones en multiples regiones geograficas.

Desventajas: - **Dependencia del proveedor (vendor lock-in)**: migrar de un proveedor a otro puede ser complejo y costoso si se usan servicios especificos del proveedor. - **Costos a largo plazo**: el modelo OpEx puede resultar mas caro que on-premises para cargas de trabajo estables y predecibles. - **Cumplimiento normativo**: para algunas industrias, alojar datos en infraestructura de terceros puede ser problematico. - **Latencia variable**: la comunicacion con sistemas internos on-premises introduce latencia adicional.

2.4.3 Modelo hibrido

El modelo **hibrido** combina infraestructura on-premises con servicios en la nube publica. No es simplemente tener ambos entornos por separado, sino **integrarlos** de forma que las cargas de trabajo puedan distribuirse estrategicamente entre ambos.

Este es el modelo adoptado en el caso de estudio de esta tesis: **OpenShift on-premises** para las cargas de trabajo principales, y **ARO (Azure Red Hat OpenShift)** para workloads especificos en la nube.

Ventajas: - **Flexibilidad**: permite elegir donde ejecutar cada carga de trabajo segun sus requisitos (regulatorios, de rendimiento, de costos). - **Migracion progresiva**: no requiere un cambio total donde todo se mueve a la nube de golpe. Permite una transicion gradual. - **Optimizacion de costos**: las cargas estables corren on-premises (costo predecible), mientras que las cargas variables aprovechan la elasticidad del cloud. - **Redundancia**: tener infraestructura en dos ubicaciones distintas mejora la resiliencia ante fallas.

Desventajas: - **Complejidad operativa**: mantener dos entornos requiere herramientas y procesos que funcionen de forma consistente en ambos. - **Networking**: la conectividad entre on-premises y cloud agrega complejidad (VPNs, ExpressRoute, latencia). - **Consistencia**: asegurar que las politicas de seguridad, monitoreo y despliegue sean las mismas en ambos entornos es un desafio constante.

2.4.3.1 ARO - Azure Red Hat OpenShift

ARO (Azure Red Hat OpenShift) es un servicio gestionado de OpenShift que corre sobre la infraestructura de Microsoft Azure. Es operado conjuntamente por Microsoft y Red Hat.

La diferencia clave con un OpenShift on-premises es que en ARO: - El **plano de control** (masters) es gestionado por Microsoft y Red Hat. La empresa no necesita preocuparse por actualizaciones ni disponibilidad de estos componentes. - El **hardware** y la **red subyacente** son de Azure. - El usuario se enfoca en desplegar y operar sus aplicaciones, no en administrar la infraestructura del

cluster.

ARO permite que una empresa que ya usa OpenShift on-premises extienda su plataforma a la nube sin cambiar sus herramientas, manifiestos ni procesos de trabajo. Un manifiesto que funciona en OpenShift on-premises deberia funcionar igual en ARO, lo que facilita la consistencia en un modelo hibrido.

2.5 Conceptos fundamentales de desarrollo y operaciones

Antes de abordar las practicas de entrega de software (CI/CD, GitOps), es necesario comprender algunos conceptos que son transversales a todo el proceso de modernizacion.

2.5.1 Control de versiones y Git

Un **sistema de control de versiones** es una herramienta que registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo. Permite saber **quien** cambio **que**, **cuando** y **por que**, y facilita la colaboracion entre multiples personas trabajando sobre los mismos archivos.

Git es el sistema de control de versiones mas utilizado en la industria del software. Fue creado en 2005 por Linus Torvalds (creador de Linux) y es de codigo abierto.

Conceptos clave de Git:

- **Repositorio (repo)**: un directorio cuyo historial de cambios esta rastreado por Git. Puede ser local (en la computadora del desarrollador) o remoto (en un servidor como GitHub, GitLab o Bitbucket).
- **Commit**: una “foto” del estado de los archivos en un momento dado. Cada commit tiene un

identificador unico, un autor, una fecha y un mensaje que describe el cambio.

- **Branch (rama)**: una linea de desarrollo independiente. Permite trabajar en una funcionalidad nueva sin afectar el codigo principal. Cuando el trabajo esta listo, se fusiona (merge) con la rama principal.
- **Merge**: la accion de combinar los cambios de una rama con otra.
- **Pull Request (PR) / Merge Request (MR)**: mecanismo de revision donde un desarrollador propone cambios y otros miembros del equipo los revisan antes de integrarlos. Es una practica fundamental de calidad de software.
- **Tag**: una marca que señala un punto especifico del historial, tipicamente usada para marcar versiones de release (v1.0, v2.1, etc.).

Git no es solo para codigo fuente. En el contexto de esta tesis, Git se utiliza tambien para versionar **infraestructura**: manifiestos de Kubernetes, configuraciones de Argo CD, definiciones de Helm charts. Este uso de Git como fuente de verdad para la infraestructura es la base de **GitOps**, que se describe en la seccion 2.6.2.

2.5.2 YAML

YAML (acronimo recursivo de "YAML Ain't Markup Language") es un formato de serializacion de datos diseñado para ser facilmente legible por humanos. Es el formato estandar para definir configuraciones en Kubernetes, OpenShift, Ansible, Docker Compose y muchas otras herramientas.

Un ejemplo simple de YAML:

```yaml
aplicacion:
```

```yaml
nombre: mi-app
version: "1.2.3"
replicas: 3
puertos:
  - 8080
  - 8443
configuracion:
  debug: false
  log_level: "info"
```

Caracteristicas de YAML: - Usa **indentacion** (espacios) para definir la jerarquia, similar a como un indice de un libro usa sangrias. - Soporta tipos de datos simples: textos, numeros, booleanos, listas. - Es sensible a la indentacion: un error de espacios puede cambiar el significado del archivo. - Se prefiere sobre otros formatos como JSON o XML por su legibilidad.

En el contexto de esta tesis, practicamente toda la configuracion de la plataforma se define en archivos YAML: deployments, services, configuraciones de Argo CD, Helm values, playbooks de Ansible. Cuando en los capitulos siguientes se muestren ejemplos de configuracion, estaran escritos en YAML.

2.5.3 Entornos de despliegue

En el desarrollo de software profesional, las aplicaciones no se despliegan directamente en produccion. Se utilizan multiples **entornos** que permiten validar los cambios progresivamente:

- **Desarrollo (dev)**: entorno donde los desarrolladores prueban sus cambios durante el desarrollo.

Suele ser inestable y actualizado con frecuencia.

- **Testing / QA**: entorno donde se ejecutan pruebas formales (funcionales, de integracion, de rendimiento) antes de avanzar.
- **Staging / Pre-produccion**: entorno que replica las condiciones de produccion lo mas fielmente posible. Es el ultimo paso antes de produccion.
- **Produccion (prod)**: el entorno real donde los usuarios finales usan la aplicacion. Cualquier error aqui impacta directamente al negocio.

La existencia de multiples entornos introduce un desafio: **mantener la consistencia entre ellos**. Una aplicacion puede funcionar perfectamente en desarrollo y fallar en produccion si las configuraciones son diferentes. Los contenedores (misma imagen en todos los entornos) y las herramientas de gestion de configuracion (Helm, Kustomize) ayudan a resolver este problema.

2.5.4 DevOps

DevOps no es una herramienta ni un producto: es una **cultura y conjunto de practicas** que buscan unificar el desarrollo de software (Dev) y las operaciones de TI (Ops), dos areas que historicamente trabajaron de forma separada y muchas veces en conflicto.

En el modelo tradicional: - Los **desarrolladores** quieren entregar cambios rapido, agregar funcionalidades, innovar. - Los **operadores** quieren estabilidad, minimizar riesgos, evitar cambios que puedan causar incidentes.

Estos objetivos contrapuestos generan fricciones: los desarrolladores piden despliegues que los operadores retrasan; los operadores piden que los desarrolladores no cambien tanto; y el negocio sufre porque la entrega de valor se ralentiza.

DevOps propone eliminar este silo mediante:

- **Responsabilidad compartida**: los equipos son responsables de todo el ciclo de vida, desde el codigo hasta la operacion en produccion.
- **Automatizacion**: todo lo que se pueda automatizar se automatiza (builds, tests, deploys, monitoreo).
- **Colaboracion**: herramientas y procesos compartidos entre desarrollo y operaciones.
- **Feedback rapido**: monitoreo, alertas y metricas que permiten detectar y corregir problemas rapidamente.
- **Mejora continua**: iteracion constante sobre los procesos, buscando ser mas eficientes y confiables.

El proceso de modernizacion descrito en esta tesis es, en esencia, la **implementacion practica de una cultura DevOps**: pasar de procesos manuales y equipos aislados a una plataforma compartida con automatizacion, estandarizacion y responsabilidad distribuida.

2.5.5 Monolitos y microservicios

La **arquitectura** de una aplicacion determina como esta organizada internamente y tiene un impacto directo en como se despliega y opera.

Monolito: una aplicacion donde toda la funcionalidad esta contenida en un unico programa. Se compila, despliega y escala como una sola unidad. La mayoria de las aplicaciones tradicionales en IIS son monolitos.

- **Ventajas**: simplicidad de desarrollo y despliegue (un solo artefacto), comunicacion interna rapida (todo corre en el mismo proceso).

- **Desventajas**: un cambio pequeño requiere redesplegar toda la aplicacion; escalar significa replicar todo el monolito aunque solo un componente necesite mas capacidad; el codigo tiende a crecer y acoplarse con el tiempo.

Microservicios: la aplicacion se descompone en multiples servicios pequeños e independientes, cada uno responsable de una funcionalidad especifica. Cada microservicio se despliega, escala y actualiza de forma independiente.

- **Ventajas**: cada servicio puede usar la tecnologia mas adecuada; los equipos pueden trabajar de forma independiente; se escala solo lo que se necesita.
- **Desventajas**: mayor complejidad operativa (hay mas cosas que gestionar); la comunicacion entre servicios es por red (mas lenta que dentro de un proceso); requiere herramientas de observabilidad y gestion mas sofisticadas.

Es importante aclarar que **contenedores no implican microservicios**. Un monolito puede (y muchas veces deberia) correr en un contenedor, obteniendo beneficios de portabilidad y consistencia sin necesidad de descomponerlo. La decision de migrar a microservicios es independiente de la decision de adoptar contenedores.

En el caso de estudio de esta tesis, la migracion a contenedores no requiere necesariamente convertir las aplicaciones en microservicios. Muchas aplicaciones se “containerizan” tal como estan (monolitos en contenedores), y la decision de descomponerlas es independiente y futura.

2.6 Practicas de entrega de software

Con los conceptos fundamentales establecidos, esta seccion describe las practicas que transforman la forma en que el software se construye, prueba y despliega.

2.6.1 Integracion Continua y Entrega Continua (CI/CD)

CI/CD es un conjunto de practicas que buscan automatizar y acelerar el ciclo de vida de la entrega de software.

Integracion Continua (CI - Continuous Integration) consiste en que los desarrolladores integren sus cambios de codigo en un repositorio compartido de forma frecuente (idealmente varias veces al dia). Cada integracion dispara un proceso automatizado que:

1. Compila el codigo.
2. Ejecuta pruebas automatizadas (unitarias, de integracion).
3. Genera artefactos (binarios, imagenes de contenedores).
4. Reporta el resultado al equipo.

El objetivo es **detectar problemas tempranamente**. Si un cambio rompe algo, se detecta en minutos, no semanas despues cuando alguien intenta hacer un despliegue.

Entrega Continua (CD - Continuous Delivery) extiende la CI asegurando que el software este **siempre en un estado desplegable**. Cada cambio que pasa las pruebas automatizadas puede ser desplegado a produccion en cualquier momento, aunque la decision final de desplegar puede ser manual (un boton de “aprobar”).

Despliegue Continuo (CD - Continuous Deployment) va un paso mas alla: cada cambio

que pasa las pruebas se despliega **automaticamente** a produccion sin intervencion humana. Este nivel requiere un alto grado de confianza en las pruebas automatizadas y en la capacidad de detectar y revertir problemas rapidamente.

2.6.1.1 Pipelines

Un **pipeline** es la implementacion concreta de CI/CD: una secuencia de etapas automatizadas que se ejecutan en orden. Un pipeline tipico incluye:

```
Codigo → Build → Test → Analisis → Imagen → Deploy (staging) → Deploy (produccion)
```

Cada etapa puede incluir validaciones que actuan como "puertas" (gates): si las pruebas fallan, el pipeline se detiene y no se despliega. Esto garantiza que solo codigo validado llegue a produccion.

Los pipelines pueden implementarse con diversas herramientas: Jenkins, Tekton, GitHub Actions, GitLab CI, Azure DevOps Pipelines, entre otras. La eleccion de la herramienta depende del contexto tecnologico de la organizacion.

2.6.2 GitOps

GitOps es un modelo operativo que usa **Git como unica fuente de verdad** para la infraestructura y la configuracion de las aplicaciones. Fue formalizado por Weaveworks en 2017, aunque los principios que lo sustentan existian antes.

La idea central es simple pero poderosa: **todo lo que esta desplegado en el cluster debe estar definido en un repositorio Git. Si algo no esta en Git, no deberia existir en el cluster.**

2.6.2.1 Principios fundamentales

1. **Declarativo**: toda la infraestructura y configuracion se define de forma declarativa (archivos YAML, Helm charts, Kustomize overlays), no con scripts imperativos que ejecutan pasos.
2. **Versionado en Git**: el estado deseado de todo el sistema se almacena en un repositorio Git. Git actua como registro de auditoria, fuente de verdad y mecanismo de colaboracion.
3. **Automatizado**: los cambios aprobados en Git se aplican automaticamente al entorno objetivo. No se realizan cambios manuales directamente en el cluster.
4. **Reconciliacion continua**: un agente observa constantemente si el estado real del cluster coincide con el estado deseado en Git. Si hay diferencias (drift), las corrige automaticamente.

2.6.2.2 Por que Git como fuente de verdad

Git no fue elegido arbitrariamente. Tiene propiedades que lo hacen ideal para gestionar infraestructura:

- **Historial completo**: cada cambio queda registrado con autor, fecha y mensaje. Esto constituye un log de auditoria natural. Si alguien pregunta "¿quien cambio la configuracion de produccion el martes?", la respuesta esta en el historial de Git.
- **Branching y merging**: permite trabajar en cambios de forma aislada y revisarlos antes de aplicarlos (Pull Requests).
- **Revertibilidad**: si un cambio causa problemas, revertir a un estado anterior es tan simple como un `git revert`. El cluster vuelve al estado previo automaticamente.
- **Colaboracion**: multiples personas pueden proponer, revisar y aprobar cambios de forma controlada.

2.6.2.3 Push vs Pull model

Existen dos modelos para aplicar los cambios desde Git al cluster:

- **Push model**: un sistema externo (pipeline, CI/CD) se conecta al cluster y ejecuta los cambios. Requiere que el sistema externo tenga credenciales de acceso al cluster, lo que amplifica la superficie de ataque.
- **Pull model**: un agente que corre **dentro** del cluster observa el repositorio Git y aplica los cambios cuando detecta diferencias. No se requiere acceso externo al cluster, lo que mejora la seguridad.

Argo CD, la herramienta adoptada en el caso de estudio de esta tesis, opera con el **pull model**.

2.6.2.4 GitOps vs despliegue manual: un ejemplo

Para ilustrar la diferencia, consideremos un cambio de version de una aplicacion:

Sin GitOps (manual): 1. Alguien se conecta al cluster con `oc` o `kubectl`. 2. Ejecuta un comando para cambiar la imagen del deployment. 3. Nadie mas sabe que se hizo el cambio ni por que. 4. Si algo falla, no hay registro claro de que se cambio.

Con GitOps: 1. Un desarrollador modifica el archivo de manifiesto en Git (cambia la version de la imagen). 2. Crea un Pull Request. Otro miembro del equipo lo revisa. 3. Se aprueba y se mergea a la rama principal. 4. Argo CD detecta el cambio y lo aplica al cluster automaticamente. 5. Todo queda registrado: quien lo pidio, quien lo aprobo, cuando se aplico.

2.6.3 Infraestructura como Codigo (IaC)

Infraestructura como Codigo (Infrastructure as Code - IaC) es la practica de gestionar y aprovisionar infraestructura a traves de archivos de definicion legibles por maquinas, en lugar de procesos manuales o herramientas interactivas.

La idea central es: **si la infraestructura se define en codigo, se puede versionar, revisar, testear y reproducir como cualquier otro software**.

Para entender por que esto es importante, consideremos el escenario opuesto: un servidor configurado manualmente a lo largo de meses, donde diferentes personas instalaron software, cambiaron configuraciones y aplicaron parches. Nadie sabe exactamente que hay instalado ni como recrearlo. Si el servidor falla, reconstruirlo es un proceso largo y propenso a errores. Este problema se conoce como **"servidor snowflake"** (copo de nieve): cada servidor es unico e irrepetible.

IaC elimina los servidores snowflake al definir todo en archivos versionados:

- **Reproducibilidad**: un entorno se puede recrear exactamente a partir de su definicion en codigo.
- **Consistencia**: se elimina la variabilidad introducida por procedimientos manuales (configuration drift).
- **Auditabilidad**: los cambios quedan registrados en el historial del repositorio.
- **Velocidad**: provisionar un nuevo entorno pasa de horas o dias a minutos.
- **Documentacion viva**: el codigo es la documentacion. Si quieres saber que hay desplegado, miras el repositorio.

En el contexto de Kubernetes y OpenShift, IaC se materializa en los **manifiestos** (archivos

YAML) que definen los recursos del cluster, y en herramientas como **Helm** y **Kustomize** que permiten gestionarlos de forma escalable.

2.7 Herramientas clave

2.7.1 Argo CD

Argo CD es una herramienta de entrega continua declarativa para Kubernetes que implementa los principios de GitOps. Es un proyecto graduado de la CNCF (Cloud Native Computing Foundation) y se ha convertido en el estandar de facto para GitOps en ecosistemas Kubernetes.

2.7.1.1 Como funciona

Argo CD se instala como un conjunto de componentes dentro del cluster de Kubernetes/OpenShift. Su funcionamiento se basa en el concepto de **Application**:

1. Se define una **Application** en Argo CD que especifica:
    - Un **repositorio Git** que contiene los manifiestos.
    - Un **path** (carpeta) dentro del repositorio.
    - Un **cluster y namespace** de destino.
2. Argo CD **observa** continuamente el repositorio Git.
3. Cuando detecta una diferencia entre lo que dice Git y lo que esta en el cluster, marca la aplicacion como **OutOfSync** (fuera de sincronizacion).
4. Dependiendo de la configuracion, puede **sincronizar automaticamente** (aplicar los cambios) o esperar aprobacion manual.

5. Si alguien modifica un recurso directamente en el cluster sin pasar por Git, Argo CD detecta el **drift** (desvio) y puede revertirlo.

2.7.1.2 Conceptos principales

- **Application**: recurso que representa una aplicacion gestionada por Argo CD. Es la unidad basica de trabajo.
- **AppProject**: agrupacion logica de Applications que permite definir politicas de acceso: que repositorios se pueden usar, a que clusters se puede desplegar, que tipos de recursos se permiten. Util para que cada equipo tenga su propio ambito controlado.
- **Sync**: accion de aplicar los cambios de Git al cluster.
- **Health Status**: Argo CD evalua si los recursos desplegados estan funcionando correctamente (Healthy, Degraded, Progressing, Missing, etc.).
- **Sync Status**: indica si el estado del cluster coincide con Git (Synced vs OutOfSync).

2.7.1.3 Interfaz y usabilidad

Argo CD provee: - **Interfaz web**: permite ver el estado de todas las aplicaciones, su arbol de recursos, logs, eventos y diferencias entre el estado deseado y el real. Es especialmente util para visualizar relaciones entre recursos. - **CLI (`argocd`)**: herramienta de linea de comandos para gestionar aplicaciones desde la terminal. - **API REST y gRPC**: para integracion con otras herramientas y automatizacion.

2.7.2 Helm

Helm es un gestor de paquetes para Kubernetes, frecuentemente descrito como “el apt/yum de Kubernetes”. Permite empaquetar, distribuir y gestionar aplicaciones de Kubernetes como unidades reutilizables llamadas **charts**.

Para entenderlo con una analogia: instalar una aplicacion en Kubernetes sin Helm es como construir un mueble sin instrucciones, cortando cada pieza manualmente. Helm es como comprar el mueble con todas las piezas y un manual, donde solo hay que elegir el color y el tamaño.

2.7.2.1 El problema que resuelve

Desplegar una aplicacion en Kubernetes tipicamente requiere multiples archivos YAML: un Deployment, un Service, un ConfigMap, un Ingress, etc. Cuando se tienen multiples entornos (desarrollo, staging, produccion) o multiples aplicaciones con configuraciones similares, gestionar estos archivos manualmente se vuelve repetitivo y propenso a errores.

Helm resuelve esto con:

- **Charts**: paquetes que contienen todos los manifiestos necesarios para desplegar una aplicacion, con la capacidad de parametrizarlos.
- **Templates**: los manifiestos dentro de un chart usan el lenguaje de templates de Go, lo que permite generar YAML dinamicamente en funcion de valores de entrada.
- **Values**: archivo (`values.yaml`) que define los parametros de configuracion de un chart. Cada entorno puede tener su propio archivo de values, manteniendo una unica base de templates.
- **Releases**: cada instalacion de un chart en un cluster es un “release” con nombre, version y

configuracion especifica.

2.7.2.2 Estructura de un Helm chart

```
mi-aplicacion/
|-- Chart.yaml          # Metadatos del chart (nombre, version, descripcion)
|-- values.yaml         # Valores por defecto
|-- templates/          # Manifiestos con templates
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- configmap.yaml
|   |-- ingress.yaml
|-- charts/             # Dependencias (sub-charts)
```

2.7.2.3 Ejemplo conceptual

En lugar de tener un Deployment con valores fijos para cada entorno:

```yaml
# Sin Helm - valores fijos para un entorno
replicas: 3
image: mi-app:1.2.3
```

Con Helm, se parametriza:

```yaml
# Con Helm - template (el "molde")
replicas: {{ .Values.replicas }}
image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
```

Y los valores se definen por entorno:

```yaml
# values-produccion.yaml
replicas: 3
image:
  repository: mi-app
  tag: 1.2.3


# values-desarrollo.yaml
replicas: 1
image:
  repository: mi-app
  tag: latest
```

Esto permite mantener una unica definicion de la aplicacion (el chart) y variar solo lo que cambia entre entornos (los values). Si se necesita cambiar algo comun a todos los entornos (por ejemplo, agregar un health check), se modifica el template una sola vez.

2.7.3 Kustomize

Kustomize es una herramienta nativa de Kubernetes para personalizar manifiestos YAML **sin usar templates**. A diferencia de Helm, Kustomize trabaja directamente sobre manifiestos YAML validos y los modifica mediante parches y superposiciones.

2.7.3.1 Enfoque: bases y overlays

El modelo de Kustomize se basa en dos conceptos:

- **Base**: los manifiestos originales de la aplicacion, exactamente como son. Son YAML valido y funcional por si solo.
- **Overlays**: modificaciones especificas para un entorno o caso de uso, que se aplican **sobre** la base sin modificarla.

```
mi-aplicacion/
|-- base/                          # Definicion base (comun a todos)
|   |-- kustomization.yaml
|   |-- deployment.yaml
|   |-- service.yaml
|-- overlays/
|   |-- desarrollo/
|   |   |-- kustomization.yaml    # Modifica replicas=1
|   |-- produccion/
|       |-- kustomization.yaml    # Modifica replicas=3, agrega recursos
```

La ventaja de este enfoque es que los archivos base son YAML valido que se puede aplicar directamente, sin necesidad de un motor de templates. Los overlays solo definen las diferencias.

2.7.3.2 Helm vs Kustomize

Ambas herramientas resuelven el problema de gestionar manifiestos para multiples entornos, pero con filosofias diferentes:

Aspecto	Helm	Kustomize
Enfoque	Templates parametrizados	Parches sobre YAML valido
Complejidad	Mayor (lenguaje de templates Go)	Menor (YAML puro)
Curva de aprendizaje	Media-alta	Baja
Ecosistema	Amplio catalogo de charts publicos	Integrado en `kubectl`
Reutilizacion	Alta (charts como paquetes)	Media (bases compartidas)
Caso de uso ideal	Paquetes reutilizables, distribucion	Personalizacion por entorno

En la practica, **Helm y Kustomize no son mutuamente excluyentes**. Muchas organizaciones usan Helm para empaquetar aplicaciones y Kustomize para personalizar los manifiestos resultantes por entorno. Argo CD soporta ambas herramientas de forma nativa.

2.7.4 Red Hat Developer Hub

Red Hat Developer Hub es la distribucion empresarial de **Backstage**, un framework de codigo abierto creado por Spotify para construir portales de desarrollo internos (Internal Developer Portals - IDP).

2.7.4.1 El problema que resuelve

A medida que una organizacion crece en equipos y servicios, surgen problemas de descubrimiento y autonomia:

- “¿Donde esta la documentacion de este servicio?”
- “¿Como creo un nuevo proyecto que cumpla con los estandares de la empresa?”
- “¿Quien es responsable de este componente?”
- “¿Que version esta desplegada en produccion?”
- “¿Que pasos tengo que seguir para que mi aplicacion tenga pipeline, repositorio y namespace?”

Sin un portal centralizado, los desarrolladores pierden tiempo buscando informacion dispersa en wikis, emails y preguntando a colegas. Los equipos de plataforma se convierten en un cuello de botella porque cada nuevo proyecto requiere su intervencion manual.

2.7.4.2 Capacidades principales

- **Catalogo de software**: registro centralizado de todos los servicios, librerias y componentes de la organizacion, con informacion de ownership (quien es responsable), documentacion y estado.
- **Software Templates**: plantillas que permiten a los desarrolladores crear nuevos proyectos con la estructura, pipelines, manifiestos y configuraciones estandar de la empresa, sin necesidad de conocer todos los detalles de la plataforma. Un template podria, por ejemplo, crear un repositorio en Git, configurar un pipeline, generar los manifiestos de Kubernetes y registrar la aplicacion en Argo CD, todo con unos pocos clicks.
- **Plugins**: arquitectura extensible que permite integrar herramientas existentes (Argo CD, Kubernetes, Git, CI/CD, monitoreo) en una unica interfaz.
- **TechDocs**: documentacion tecnica integrada, generada a partir de archivos Markdown en los repositorios de codigo.

2.7.4.3 Rol en la modernizacion

En el contexto de esta tesis, Developer Hub tiene un rol especifico: **facilitar el onboarding de nuevos equipos a la plataforma OpenShift**. En lugar de que cada equipo tenga que aprender desde cero como crear namespaces, configurar pipelines, definir manifiestos y conectar con Argo CD, un Software Template automatiza todo eso.

Esto transforma el proceso de onboarding de "hablar con el equipo de plataforma y esperar dias" a "usar el portal y tener todo listo en minutos".

2.7.5 Ansible

Ansible es una herramienta de automatizacion de TI desarrollada por Red Hat. Se utiliza para gestionar configuraciones, desplegar aplicaciones, orquestar tareas complejas y aprovisionar infraestructura.

2.7.5.1 Caracteristicas principales

- **Sin agentes (agentless)**: Ansible no requiere instalar software en las maquinas que gestiona. Se conecta via SSH (Linux) o WinRM (Windows) y ejecuta las tareas remotamente.
- **Lenguaje declarativo (YAML)**: las tareas se definen en archivos YAML llamados **playbooks**, que describen el estado deseado de un sistema.
- **Idempotencia**: ejecutar un playbook multiples veces produce el mismo resultado. Si el sistema ya esta en el estado deseado, Ansible no hace cambios. Esto es fundamental para la confiabilidad: ejecutar una automatizacion "de mas" no causa problemas.
- **Inventarios**: listas de hosts (servidores) agrupados que definen sobre que maquinas se ejecutan

las tareas.

- **Roles y Collections**: mecanismos de empaquetado y reutilizacion de automatizaciones, similares en concepto a las librerias de software.

2.7.5.2 Ejemplo de playbook

```
- name: Configurar servidor web
  hosts: webservers
  tasks:
    - name: Instalar nginx
      package:
        name: nginx
        state: present       # "Quiero que este instalado"


    - name: Iniciar servicio nginx
      service:
        name: nginx
        state: started        # "Quiero que este corriendo"
        enabled: true         # "Quiero que arranque con el servidor"
```

Este playbook es legible incluso para alguien sin experiencia en Ansible: se lee como una lista de tareas en lenguaje casi natural.

2.7.5.3 Ansible en el contexto de OpenShift

En el caso de estudio, Ansible se utiliza para tareas que estan **fuera** del alcance de Kubernetes y Argo CD:

- **Automatizacion de tareas de plataforma**: configuracion inicial de clusters, gestion de usuarios y permisos, aplicacion de politicas de seguridad.
- **Aprovisionamiento de infraestructura**: creacion de recursos que no son propios de Kubernetes (maquinas virtuales, configuraciones de red, integraciones con sistemas externos).
- **Operaciones repetitivas**: tareas de mantenimiento, actualizaciones, backups que se ejecutan de forma programada.
- **Integracion con sistemas legacy**: automatizacion de tareas sobre sistemas Windows, IIS u otros componentes que conviven con la nueva plataforma durante la transicion.

Ansible complementa a Argo CD cubriendo el espacio de automatizacion **fuera** del cluster, donde las herramientas de GitOps no llegan.

2.8 Patrones de arquitectura

2.8.1 Patron App of Apps

El patron **App of Apps** es un patron de gestion declarativa de aplicaciones en Argo CD. Resuelve el problema de escalar GitOps cuando se tienen muchas aplicaciones.

2.8.1.1 El problema

Cuando una organizacion tiene pocas aplicaciones, crear un recurso Application de Argo CD para cada una es manejable. Pero cuando se tienen decenas o cientos de aplicaciones, gestionarlas individualmente genera preguntas:

- ¿Quien crea las Applications? ¿Manualmente en la interfaz de Argo CD?
- ¿Donde se definen? ¿En la cabeza de alguien?
- ¿Como se asegura la consistencia entre ellas?
- ¿Como se agregan nuevas aplicaciones sin intervencion manual del equipo de plataforma?

2.8.1.2 La solucion

El patron consiste en crear una **Application "raiz"** que, en lugar de apuntar a los manifiestos de una aplicacion de negocio, apunta a un directorio que contiene **definiciones de otras Applications**. Es decir, una Application que gestiona Applications.

```
repo-de-apps/
|-- apps/                          # La app raiz apunta aca
|   |-- app-frontend.yaml          # Application de Argo CD para el frontend
|   |-- app-backend.yaml           # Application de Argo CD para el backend
|   |-- app-api.yaml               # Application de Argo CD para la API
|   |-- app-monitoring.yaml        # Application de Argo CD para monitoreo
```

Cuando Argo CD sincroniza la app raiz, crea (o actualiza) todas las Applications hijas. Cada Application hija a su vez apunta al repositorio Git correspondiente con los manifiestos de su

aplicacion.

2.8.1.3 Beneficios

- **Gestion centralizada**: todas las aplicaciones se definen en un unico lugar versionado.
- **Automatizacion**: agregar una nueva aplicacion es tan simple como agregar un archivo YAML al directorio y hacer un commit. Argo CD la detecta y la crea automaticamente.
- **Consistencia**: las politicas (sync policy, project, destino) se pueden estandarizar en los templates.
- **Escalabilidad**: funciona igual con 5 aplicaciones que con 500.
- **Auditabilidad**: el historial de Git muestra cuando se agrego, modifico o elimino cada aplicacion.

2.8.1.4 Variantes

El patron tiene variantes segun la complejidad de la organizacion:

- **App of Apps simple**: una app raiz que gestiona todas las apps de un cluster.
- **App of Apps por entorno**: una app raiz por entorno (desarrollo, staging, produccion), cada una apuntando a un directorio o branch diferente.
- **App of Apps jerarquico**: una app raiz que gestiona apps intermedias (por equipo o por area), que a su vez gestionan las apps finales. Util para organizaciones grandes con multiples equipos y clusters.

2.8.2 Operadores de Kubernetes

Los **Operadores** son un patron de Kubernetes que extiende la plataforma para gestionar aplicaciones complejas de forma automatizada. Encapsulan conocimiento operativo (que normalmente estaria en la cabeza de un administrador experimentado) en software que corre dentro del cluster.

2.8.2.1 El problema que resuelven

Algunas aplicaciones son mas complejas de operar que otras. Desplegar un servicio web stateless (sin estado) es relativamente simple: se crea un Deployment, un Service y listo. Pero gestionar una base de datos en cluster, un broker de mensajes o un sistema de monitoreo requiere conocimiento especializado: como configurar replicas, como hacer backups, como recuperarse de fallos, como actualizarse sin perder datos.

2.8.2.2 Como funcionan

Un operador tipicamente:

1. Define un **Custom Resource (CR)**: un nuevo tipo de recurso en Kubernetes que representa el servicio. Por ejemplo, un CR `PostgresCluster` que define "quiero un cluster de PostgreSQL con 3 replicas y backups diarios".
2. Implementa un **controller**: un programa que observa los CRs y toma acciones para alcanzar y mantener el estado deseado.
3. Automatiza operaciones complejas: instalacion, actualizacion, backup, recuperacion ante fallos, escalado, rotacion de credenciales, etc.

2.8.2.3 Ejemplo

Un operador de base de datos podria: - Crear automaticamente un cluster de PostgreSQL cuando el usuario crea un CR `PostgresCluster`. - Gestionar replicas y failover automatico sin intervencion manual. - Realizar backups programados y almacenarlos en un destino configurado. - Actualizarse a nuevas versiones de forma controlada (rolling update).

OpenShift hace uso extensivo de operadores: - Para sus **propios componentes internos** (networking, monitoreo, registry, autenticacion). - Para **aplicaciones de terceros** disponibles a traves del **OperatorHub** (bases de datos, herramientas de monitoreo, integraciones, etc.).

2.9 Resumen del capitulo

Los conceptos presentados en este capitulo forman la base teorica sobre la que se construye el proceso de modernizacion descrito en esta tesis. La siguiente tabla resume cada concepto y su relevancia:

Concepto	Relevancia en la tesis
IIS / Infraestructura tradicional	Punto de partida: la plataforma que se busca modernizar
Maquinas virtuales	Contexto de la infraestructura existente
Contenedores	Unidad fundamental de despliegue en la nueva plataforma

Concepto	Relevancia en la tesis
Kubernetes / OpenShift	Plataforma de orquestacion que reemplaza el modelo IIS
Modelo hibrido (on-prem + ARO)	Arquitectura de despliegue adoptada
Git	Herramienta de versionado; base de GitOps
DevOps	Cultura y practicas que guian la transformacion
CI/CD	Automatizacion del ciclo de build y deploy
GitOps	Modelo operativo basado en Git como fuente de verdad
Infraestructura como codigo	Definicion declarativa de toda la plataforma
Argo CD	Herramienta que implementa GitOps en el cluster
Helm / Kustomize	Gestion de manifiestos para multiples entornos
Developer Hub	Portal para onboarding autonomo de equipos
Ansible	Automatizacion de tareas fuera de Kubernetes
App of Apps	Patron para escalar GitOps a muchas aplicaciones
Operadores	Extension de Kubernetes para gestionar servicios complejos

En los capitulos siguientes, se vera como estos conceptos se fueron adoptando progresivamente en la organizacion, desde los primeros pasos con comandos manuales hasta la plataforma

actual.